VEL TECH RANGARAJAN DR. SAGUNTHALA R&D INSTITUTE OF SCIENCE AND TECHNOLOGY
Chennai, India · Office of International Relations · Value Added Course

Reinforcement Learning and Language Model Alignment

VTR UGE 21

DAY 4 · MODULE 4

Reinforcement Learning for Language Models

From a language model to an aligned one: fine-tuning, preferences, a learned reward, reinforcement learning from human feedback (RLHF) with a Kullback-Leibler (KL) penalty, and what happens without it.

Prof. Dr. Utku Kose
Full Professor, Department of Computer Engineering, Süleyman Demirel University, Isparta, Türkiye
Founding Director, AI Application and Research Center (YAZEM), Süleyman Demirel University
Head of the Computer Science Division, Department of Computer Engineering, Süleyman Demirel University
Additional affiliations: University of North Dakota (USA), Universidad Panamericana (Mexico City, Mexico), Vel Tech University (Chennai, India)
IEEE Senior Member, ACM Professional Member

ORCID 0000-0002-9652-6415 · utkukose.com · github.com/utkukose
utkukose@sdu.edu.tr · utku.kose@und.edu · ukose@up.edu.mx · utkukose@gmail.com

Overview

Large language models are aligned with human preferences in three stages: supervised fine-tuning on good examples, a reward model learned from comparisons made by people, and reinforcement learning that maximises this reward while staying close to the fine-tuned model [1, 2, 3]. The procedure is called reinforcement learning from human feedback, abbreviated RLHF. This day builds every stage on a small world of sentences, with a language model written in NumPy [4]. Because the true quality of every sentence is known, the day can measure what each stage achieves and what goes wrong when the penalty that keeps the model close to its start, a penalty on the Kullback-Leibler (KL) divergence, is removed [5, 6].

Estimated study time: 6 to 8 hours.

Learning outcomes

By the end of the day, students are expected to train an autoregressive language model and sample from it, to verify its gradient, to generate preference pairs with a noisy annotator and fit a Bradley-Terry reward model, to explain why a held-out accuracy on noisy labels understates the quality of a reward model, to implement RLHF as REINFORCE with a KL penalty, to recognise reward hacking from the divergence of the reward-model score and the gold reward, and to choose the KL weight by measurement. In Python, students are expected to turn text into integer sequences and back, pad them into arrays and sample from a probability vector.

Python in this day

The Colab notebook opens with Python step 4: Text as integers, right after its setup section. It covers splitting text into words, building a vocabulary with special tokens, encoding and decoding, padding sequences to a fixed length, sampling the next token from a probability vector and the log-probability of a sequence [7]. After the step, the notebook continues with the language model, the reward model and RLHF.

How these notes point to the Python code. Every method of this lecture is also written as Python code in the Colab notebook of the day, the file `NB04_rl_for_language_models.ipynb`. A Colab notebook is a document of text and Python code that runs in the web browser, with nothing to install. It opens with the button Open in Colab on the day overview page of the course, https://github.com/utkukose/rl-alignment-veltech-NB-lecture/tree/main/days/day-04. The code of the notebook is divided into numbered sections. In these notes, each box titled *Python code in the Colab notebook, Section N* names the section whose code carries out what the text above the box explains, and says what to run and what to look at there. The animations that the notes mention are interactive figures on the lecture page of the day, https://utkukose.github.io/rl-alignment-veltech-NB-lecture/days/day-04/lecture.html. The interactive lab, with the practice parts and the self-assessment of the day, is at https://utkukose.github.io/rl-alignment-veltech-NB-lecture/days/day-04/lab.html. Each box titled *Interactive lab, Part X* opens one part of the lab. The part ends with a link back to the lecture page, and the notebook is always opened from the boxes of the lecture.

Python step 4: Text as integers

A language model never sees letters or words: It sees integers. Each integer stands for a token, as one unit of text is called, here a word. This step turns a sentence into a sequence of integers and back, pads it to a

fixed length, samples a next token from a probability vector and computes the log-probability of a sequence [7].

Words and a vocabulary

```
sentence = "the cat slept on the mat"
words = sentence.split()
vocab = ["<pad>", "<bos>", "<eos>"] + sorted(set(words))
stoi = {w: i for i, w in enumerate(vocab)}
print(words)
print(stoi)
```

Output

```
['the', 'cat', 'slept', 'on', 'the', 'mat']
{'<pad>': 0, '<bos>': 1, '<eos>': 2, 'cat': 3, 'mat': 4, 'on': 5, 'slept': 6, 'the': 7}
```

`split` cuts a string at spaces into a list of words. `set` removes duplicates, `sorted` orders them, and the three special tokens mark padding, the beginning and the end of a sentence. The dictionary `stoi`, string to integer, gives every token its number.

Encoding, decoding and padding

```
ids = [stoi["<bos>"]] + [stoi[w] for w in words] + [stoi["<eos>"]]
padded = ids + [stoi["<pad>"]] * (10 - len(ids))
print(ids)
print(padded)
print(" ".join(vocab[i] for i in ids[1:-1]))
```

Output

```
[1, 7, 3, 6, 5, 7, 4, 2]
[1, 7, 3, 6, 5, 7, 4, 2, 0, 0]
the cat slept on the mat
```

A list comprehension encodes every word. Padding brings every sequence to the same length, so that many sentences fit into one array. Decoding looks each integer up in the vocabulary list.

Sampling the next token

```
import numpy as np
p = np.array([0.0, 0.0, 0.1, 0.05, 0.6, 0.25])      # probabilities of six tokens
rng = np.random.default_rng(0)
draws = [int((p.cumsum() > rng.random()).argmax()) for _ in range(1000)]
print(np.bincount(draws, minlength=6) / 1000)
```

Output

```
[0.    0.    0.089 0.042 0.602 0.267]
```

`cumsum` stacks the probabilities into the intervals of a line from 0 to 1; a uniform random number falls into the interval of token k with probability p_k, and `argmax` finds that interval. A thousand draws reproduce the probabilities closely. The language model samples every token of a sentence this way.

The log-probability of a sequence

```
probs = [0.9, 0.5, 0.8, 0.95]          # model probabilities of the four chosen tokens
logp = np.log(probs).sum()
print("log-probability:", round(float(logp), 3), "  probability:",
round(float(np.exp(logp)), 3))
```

Output

```
log-probability: -1.073   probability: 0.342
```

The probability of a sequence is the product of the probabilities of its tokens, and its logarithm is a sum, which is numerically safe for long sequences. Here log 0.9 + log 0.5 + log 0.8 + log 0.95 = −1.073, with the natural logarithm `np.log`. The lecture writes this sum as $\log \pi_\theta(y)$, the log-probability of the sentence y under the model π_θ. REINFORCE, the policy gradient algorithm of Day 3, and direct preference optimisation (DPO), the method of Day 5, both work with this sum.

Quick check. Why do language-model computations use log-probabilities instead of probabilities?

A small language model

TokenWorld so far

The small TokenWorld of Days 1 to 3 has four tokens and 161 states. Day 1 solved it exactly, with the best sentence `the cat sat <eos>` and its reward of 2.75. Day 2 learned its values and its best sentence from experience, without the rules. Day 3 learned the probabilities of the tokens directly with policy gradients, and proximal policy optimisation kept each update small.

This day enlarges TokenWorld. A grammar of subjects, verbs, places and extras allows 1512 sentences, and a neural language model writes them token by token. The reward is no longer known to the learner: A simulated annotator compares pairs of sentences, a reward model learns from these comparisons, and reinforcement learning raises the score of the reward model. A gold reward, which only the simulation knows, measures what each stage achieves.



Figure. The stage of the project reached today: a neural language model aligned with preferences.

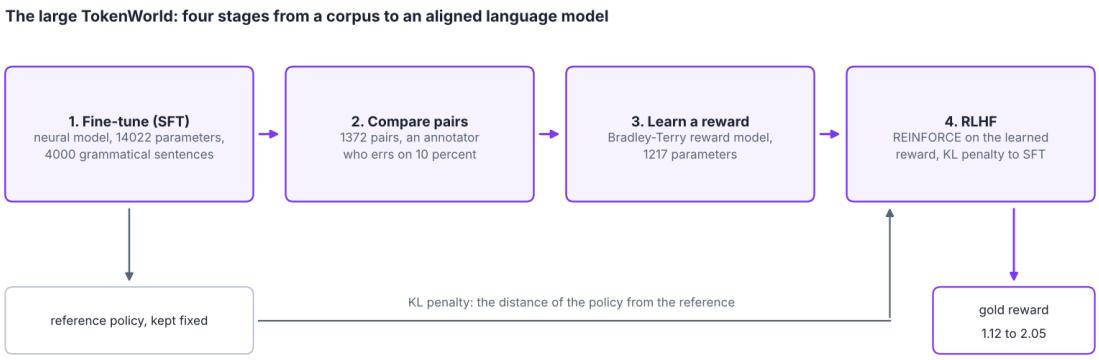


Figure. The plan of the day: four stages from a corpus of sentences to an aligned language model, with the numbers of the experiments. The sections of this lecture follow the stages from left to right.

A language model gives a probability to every possible next word, given the words so far [4]. Writing a sentence means choosing words one by one, which makes the model a policy in the sense of Day 3: The words so far are the state and the next word is the action. The project now moves to a large version of TokenWorld, a small grammar of sentences such as `the cat slept on the mat`, and to a neural language model that learns it in seconds.

> **In everyday life: a language model in the pocket**
>
> The keyboard of a mobile phone suggests the next word while a message is typed. It has learned from much text which words usually follow the words so far and offers the most probable ones. A language model does the same, with a probability for every word of its vocabulary, and writing a whole sentence means accepting one suggestion after another.

A unit of text is called a token, here a word or one of three special tokens, and the list of all tokens is the vocabulary. A finished sentence is a sequence of tokens $y = (y_1, y_2, \ldots, y_T)$ that ends with the end token `<eos>`. The policy $\pi_\theta(y_t \mid y_1, \ldots, y_{t-1})$ is the probability of the token y_t given the tokens before it, and θ stands for the parameters of the model, the numbers that training adjusts. A model that writes each token from the tokens before it is called autoregressive. The probability of a whole sentence is the product of the probabilities of its tokens, and its logarithm is a sum:

$$\pi_\theta(y) = \prod_{t=1}^{T} \pi_\theta(y_t \mid y_1, \ldots, y_{t-1})$$

$$\log \pi_\theta(y) = \sum_{t=1}^{T} \log \pi_\theta(y_t \mid y_1, \ldots, y_{t-1})$$

The sign Π multiplies the terms for $t = 1, 2, \ldots, T$, as Σ adds them, and log is the natural logarithm, the ln of Day 1. The second line is the log-probability of the sentence. It is negative or zero, and a value closer to zero means a more likely sentence.

The large TokenWorld is built from four lists of six entries each: subjects such as `the cat` or `my friend`, verbs such as `slept`, places such as `on the mat`, and extras such as `quietly` or `at dawn`. A sentence is one subject, one verb and one place, followed in 45 percent of the cases by one extra. With six extras or none, this grammar allows 6 × 6 × 6 × 7 = 1512 sentences of six to nine words. 4000 sentences drawn from it at random, with repetitions, form the corpus of the day, as a collection of example text is called. The vocabulary has 38 tokens: the 35 words of the grammar and three special tokens. `<bos>` opens every sentence, `<eos>` ends it, and `<pad>` fills shorter sentences up to a common length of 12 tokens.

Worked example: the probability of a sentence

By the rules of the grammar, the sentence `the cat slept on the mat` is written with the following probabilities. Three of the six subjects begin with `the`: 1/2. One of these three continues with `cat`: 1/3. The verb is one of six: 1/6. The place is one of six, and its first word `on` fixes the two words after it: 1/6, then 1 and 1. The sentence ends without an extra: 0.55.

The probability of the sentence is the product, 1/2 × 1/3 × 1/6 × 1/6 × 1 × 1 × 0.55. Step by step, 1/2 × 1/3 = 1/6, 1/6 × 1/6 = 1/36, 1/36 × 1/6 = 1/216 = 0.00463, and 0.00463 × 0.55 = 0.00255. Its log-probability is log 0.00255 = −5.97. The sentence has seven tokens, the end token included, so this is −5.97 / 7 = −0.85 per token.

Because this world is artificial, the quality of a sentence can be written down as a function, the gold reward. It prefers a calm, well-formed, non-repetitive and reasonably short sentence. In a real project this judgement exists only in the heads of people. Here it is known, which makes the result of every stage measurable: To align the model means to move its sentences towards what is wanted, here towards a high gold reward. The stages themselves never read the gold reward. Only the simulated annotator of the second stage consults it.

Property of the sentence	Gold reward
It starts with `the`, `a` or `my`	+1.0, otherwise −0.5
Each different calm word in it	+0.45
Each different loud word in it	−0.45
Each word beyond the sixth	−0.10
A content word occurs more than once	−0.8, counted once
The sentence has no word at all	−2.0 as the whole reward

The calm words are `quietly`, `slept`, `waited`, `beside`, `under`, `garden`, `window` and `dawn`, and the loud words are `sang`, `ran`, `happily`, `again` and `robot`. A content word is any word outside a list of twelve function words such as `the`, `a`, `in` and `on`. As a formula, with $|y|$ for the number of words of the sentence y:

$$r_{\text{gold}}(y) = o(y) + 0.45\, n_{\text{calm}}(y) - 0.45\, n_{\text{loud}}(y) - 0.10\, \max(0,\, |y| - 6) - 0.8\, \text{rep}(y)$$

Here $o(y)$ is +1.0 or −0.5 for the opening, $n_{\text{calm}}(y)$ and $n_{\text{loud}}(y)$ count the different calm and loud words, and $\text{rep}(y)$ is 1 when a content word is repeated and 0 otherwise. The term with max is zero up to six words. The gold reward does not test the grammar beyond the first word, so the share of sentences that follow the grammar is reported as a second measure.

Worked example: the gold reward of two sentences

The sentence `the cat waited under the tree quietly` starts with `the`, which gives +1.0. It contains the three calm words `waited`, `under` and `quietly`, which gives 3 × 0.45 = 1.35, and no loud word. It has seven words, one more than six, which costs 0.10. Its gold reward is 1.0 + 1.35 − 0.10 = 2.25, the highest value that a sentence of the grammar can reach.

The sentence `a robot sang near the door happily` contains the three loud words `robot`, `sang` and `happily` and also has seven words: 1.0 − 1.35 − 0.10 = −0.45, the lowest value in the grammar. Over the 1512 sentences of the grammar the mean gold reward is 1.10, and over the corpus it is 1.16.

Python code in the Colab notebook, Section 1.

Open Section 1 of the Colab notebook and run it. The function `make_corpus` draws the 4000 sentences, and the function `gold_reward` is the table above written as Python code, one line per rule. The cell prints the size of the corpus and of the vocabulary, an example sentence and the mean gold reward of the corpus.

The policy is a small neural network in the style of the neural probabilistic language model of Bengio and colleagues [4]. To compute the probabilities of the token at position t, it looks at the three tokens before it and at the position:

$$x_t = \left[E_{y_{t-3}},\ E_{y_{t-2}},\ E_{y_{t-1}},\ p_t \right]$$

$$h_t = \tanh\left(x_t W_1 + b_1\right)$$

$$z_t = h_t W_2 + b_2$$

$$\pi_\theta(v \mid y_1, \ldots, y_{t-1}) = \frac{e^{z_{t,v}}}{\sum_u e^{z_{t,u}}}$$

The table E holds a vector of 16 numbers for every token, called its embedding, and the table p one such vector for every position. The input x_t puts the embeddings of the three previous tokens and the vector of the position side by side, 64 numbers in all, with `<bos>` standing in for tokens before the start. The second line is a hidden layer of 128 units. It multiplies the input by the matrix W_1, adds the vector b_1 and applies the hyperbolic tangent tanh, which squashes every number into the range from −1 to 1. The third line gives one score $z_{t,v}$, called a logit, for each of the 38 tokens v. The last line is the softmax of Day 3, which turns the scores into probabilities that add up to one. This computation from the tokens to the probabilities is the forward pass.

The parameters θ are the 14022 entries of E, p, W_1, b_1, W_2 and b_2. They start as small random numbers, so the untrained model gives all tokens nearly the same probability. A sentence is sampled token by token: The next token is drawn at random with the probabilities of the model and appended, until `<eos>` is drawn. After ten words the sampler writes `<eos>` itself, and `<pad>` is never drawn.

Python code in the Colab notebook, Section 2.

Open Section 2 of the Colab notebook and run it. The function `forward` computes the four lines of the model above, `seq_logprob` gives the log-probability of a sentence, `grad_weighted` gives the gradient of the weighted loss, and `sample` writes sentences token by token.

Supervised fine-tuning

The first stage trains the model on examples of good behaviour. It is called supervised fine-tuning, abbreviated SFT. In a real project the model has been pretrained before this stage: It has learned to continue text from a very large corpus, and fine-tuning shows it a few thousand examples of good answers. The small model of this day is not pretrained. It starts from random parameters, and the 4000 sentences of the corpus are its examples.

Training lowers a loss, a number that is small when the model does what is wanted. Here the loss is the negative mean log-probability of the tokens of the examples, known as the cross-entropy:

$$L_{\mathrm{SFT}}(\theta) = -\frac{1}{M} \sum_{n=1}^{B} \log \pi_\theta \left(y^{(n)} \right)$$

Here $y^{(1)}$, …, $y^{(B)}$ are the B sentences of a batch, a group of examples that is processed together. The term $\log \pi_\theta(y^{(n)})$ is the log-probability of sentence n from the first section, and M is the number of tokens of the batch, end tokens included. The loss is lowered by gradient descent. The gradient $\nabla_\theta L$, written with the symbol nabla, is the list of the slopes of the loss with respect to every parameter, and a step against it lowers the loss:

$$\theta \leftarrow \theta - \alpha \, \nabla_\theta L(\theta)$$

The arrow means that the right-hand side replaces the old value, and the learning rate α sets the size of the step. The training uses a common refinement of this rule called Adam, short for adaptive moment estimation. For every parameter, Adam keeps running averages m of the slope g and v of its square, both starting at zero. Update number k is

$$m \leftarrow 0.9\, m + 0.1\, g$$

$$v \leftarrow 0.999\, v + 0.001\, g^2$$

$$\theta \leftarrow \theta - \alpha \, \frac{m / (1 - 0.9^k)}{\sqrt{v / (1 - 0.999^k)} + 10^{-8}}$$

Dividing by the square root of v gives every parameter a step of about the size α or less, whatever the scale of its slope. The divisions by $1 - 0.9^k$ and $1 - 0.999^k$ correct for the start at zero, and 10^{-8} guards against a division by zero. Before each step the gradient of the language model is clipped: If its length, the square root of the sum of its squared entries, exceeds 1, it is scaled down to length 1. Fine-tuning passes through the corpus 12 times. Each pass, called an epoch, shuffles the sentences into 31 batches of about 129, which gives 372 updates with $\alpha = 0.003$.

Every gradient of the day comes from one loss, in which each sentence has its own weight w_n:

$$L_w(\theta) = -\sum_{n=1}^{B} w_n \log \pi_\theta \left(y^{(n)} \right)$$

With $w_n = 1 / M$ this is the loss of fine-tuning, and the third stage uses the same loss with other weights.

After fine-tuning on grammatical sentences, almost every sentence the model writes follows the grammar: Of 512 sampled sentences, 98.6 percent are among the 1512 of the grammar. The mean log-probability per token on the corpus is −0.902, so the token that actually follows receives a probability of about $e^{-0.902} = 0.41$ on average. Two values give the scale. Before training the model guesses blindly among the 37 tokens that can be written, close to $\ln(1/37) = -3.61$. A model that knew the rules of the grammar exactly would reach about −0.90 and no more, because the sentences of the corpus are assembled at random.

Following the grammar is not the same as being good, however; the quality of the sentences, measured by the gold reward, stays modest. The mean gold reward of the samples is 1.12, close to the 1.16 of the corpus and far from the 2.25 of the best sentences. This model is the starting point of everything that follows, and it is kept unchanged as the reference.

> **Python code in the Colab notebook, Section 3.**
>
> Open Section 3 of the Colab notebook and run it. The function `train_sft` passes through the corpus 12 times and takes one Adam step per batch. The cell prints the mean log-probability per token, the mean gold reward of 512 sampled sentences and the share of them that follow the grammar.

Preferences and the annotator

People find it easier to compare two answers than to score one. The second stage therefore collects preferences: Two sentences are shown, and an annotator, a person who labels data, picks the better one. The result is a preference pair with a winner y_w and a loser y_l, and the recorded choice is its label. The sign $\succ$ reads *is preferred to*, so the label says $y_w \succ y_l$. In the project the annotator is simulated: It follows the gold reward but makes mistakes on a share of the pairs, as real annotators do.

> **In everyday life: comparing is easier than scoring**
>
> Asked to score a cup of tea from 1 to 10, two people give different numbers, and the same person gives another number the next day. Asked which of two cups tastes better, most people answer quickly and agree more often. Food tasting panels and eye tests at the optician use comparisons for this reason, and so do the annotators of a language model.

3000 sentences sampled from the fine-tuned model are grouped into 1500 pairs. The simulated annotator prefers the sentence with the higher gold reward. A pair with equal gold rewards is a tie and is skipped, which leaves 1372 usable pairs. With a probability η, the Greek letter eta, the annotator then names the wrong sentence. The error rate is 0.10, and 141 of the 1372 labels, 10.3 percent, come out flipped. Many pairs are close calls: In a second sample of 500 pairs the median difference of the gold rewards is 0.45, and 25.4 percent of the pairs differ by less than 0.2.

The noise sets a ceiling on how well any model can agree with the labels, as the animation shows. The accuracy of a model is the share of pairs in which it gives the higher score to the labelled winner. Suppose that a model puts a share a of the pairs in the same order as the gold reward. On these pairs it agrees with the label unless the label is flipped, and on the others only if it is flipped. Its expected accuracy on noisy labels is therefore

$$\mathrm{acc}_{\mathrm{noisy}} = (1 - \eta)\, a + \eta\,(1 - a)$$

A model with $a = 1$ reaches $1 - \eta$ and no more, 0.90 for an error rate of 0.10, and a model with $a = 0.99$ is expected to reach $0.9 \times 0.99 + 0.1 \times 0.01 = 0.891 + 0.001 = 0.892$. In the language of communication engineering, the annotator is a noisy channel that flips each label with probability η.

The animation shows the ceiling for an annotator whose preference is itself a matter of probability. Let $\Delta = r(A) - r(B)$, with the Greek capital letter delta, be the difference of the rewards of two sentences A and B. The first curve is the logistic function, also called the sigmoid:

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

It turns any number z into a number between 0 and 1: $\sigma(0) = 0.5$, large positive z give values near 1 and large negative z values near 0. The next section uses $\sigma(\Delta)$, with the Greek letter sigma, as the probability that A is preferred. The second curve is what labels with errors show of this preference. The label names A when A is preferred and no error occurs, or when B is preferred and an error occurs:

$$P_{\mathrm{label}}(A \succ B) = (1 - \eta)\,\sigma(\Delta) + \eta\,(1 - \sigma(\Delta))$$

$$= \eta + (1 - 2\eta)\,\sigma(\Delta)$$

This curve stays between η and $1 - \eta$: However clear the preference, the labels agree with it in at most a share $1 - \eta$ of the cases. With an error rate of 10 percent the animation reports a best achievable accuracy

of 90 percent. For $\Delta = 0.2$, $\sigma(0.2) = 1 / (1 + e^{-0.2}) = 1 / (1 + 0.819) = 0.550$, and the label probability is $0.1 + 0.8 \times 0.550 = 0.1 + 0.440 = 0.540$.

Animation on the lecture page: The logistic curve of the Bradley-Terry model and what labels with an error rate η show of it. Raise the error rate and read off the best accuracy any model can reach on noisy held-out pairs, $1 - \eta$.

> **Python code in the Colab notebook, Section 4.**
>
> Open Section 4 of the Colab notebook and run it. The function `make_pairs` samples the pairs, lets the annotator choose by the gold reward and flips a label with the probability `noise`. The cell prints the number of usable pairs and of flipped labels, and the median difference of the gold rewards within a pair.

> Check your understanding (1). An annotator errs on 20 percent of the pairs. A reward model puts 95 percent of the pairs in the same order as the gold reward. Which accuracy on the noisy labels is expected?
>
> A. 0.95
>
> B. 0.80
>
> C. 0.77
>
> D. 0.75

Learning a reward from comparisons

The Bradley-Terry model turns comparisons into a score [8]: The probability that sentence A is preferred to sentence B is the logistic function of the difference of their scores.

$$P(A \succ B) = \sigma\left(r(A) - r(B)\right)$$

Here $r(A)$ and $r(B)$ are the scores of the two sentences. Only their difference counts: Equal scores give a probability of 0.5, and the probability approaches 1 as the score of A rises above the score of B. Since $\sigma(-z) = 1 - \sigma(z)$, the probabilities $P(A \succ B)$ and $P(B \succ A)$ add up to 1.

The scores come from a reward model r_φ, a small neural network with parameters φ, the Greek letter phi. It describes a sentence y by 17 numbers, its features $F(y)$, and turns them into one score:

$$F(y) = \left[\frac{1}{|y|} \sum_{i=1}^{|y|} E_{y_i}, \quad \frac{|y|}{10} \right]$$

$$r_\phi(y) = \tanh\left(F(y)\, U_1 + c_1\right) \cdot u_2 + c_2$$

The first 16 features are the mean of the embeddings of the $|y|$ words, taken from an embedding table E of the reward model's own. The last feature is the number of words divided by 10. A hidden layer of 32 units with the matrix U_1 and the vector c_1 follows, and the score is the sum of its outputs weighted by the vector u_2, plus the number c_2. With the table E, these are the 1217 parameters φ. The design has a blind spot. A mean does not change when the words change places, so the reward model cannot see the order of the words. It also has no feature for repetition: A repeated word only pulls the mean towards its own embedding, which the model reads as more of that word.

A reward model is trained to make the observed preferences likely [1]. Its loss is the negative mean log-probability that the Bradley-Terry model gives to the labels of a batch of n pairs:

$$L(\phi) = -\frac{1}{n}\sum_{i=1}^{n} \log \sigma\left(\Delta_i\right)$$

$$\Delta_i = r_\phi\left(y_w^{(i)}\right) - r_\phi\left(y_l^{(i)}\right)$$

Here Δ_i is the score of the winner of pair i minus the score of its loser. The loss is small when winners receive clearly higher scores than losers. For one pair with the loss $L_i = -\log \sigma(\Delta_i)$, the slopes with respect to the two scores show what a training step does:

$$\frac{\partial L_i}{\partial r_\phi(y_w)} = -\left(1 - \sigma(\Delta_i)\right), \qquad \frac{\partial L_i}{\partial r_\phi(y_l)} = 1 - \sigma(\Delta_i)$$

A step against these slopes raises the score of the winner and lowers the score of the loser, both in proportion to $1 - \sigma(\Delta_i)$. It is close to 1 when the model puts a pair in the wrong order by a wide margin and close to 0 when the order is right by a wide margin. The last 15 percent of the pairs are set aside, 206 held-out pairs that the model never trains on. The reward model trains on the other 1166 pairs for 30 epochs, in batches of about 130 pairs, with Adam and $\alpha = 0.003$.

> **Worked example: a Bradley-Terry probability and its loss**
>
> A reward model gives sentence A the score 2.0 and sentence B the score 0.5. The difference is 2.0 − 0.5 = 1.5, so $P(A \succ B) = \sigma(1.5) = 1 / (1 + e^{-1.5})$. With $e^{-1.5} = 0.223$ this is 1 / 1.223 = 0.818, and $P(B \succ A) = 1 - 0.818 = 0.182$.
>
> If the annotator has chosen A, the loss of this pair is −log 0.818 = 0.20, a small number, because the model agrees with the label. The training step then moves the two scores only a little, in proportion to 0.182. If the annotator has chosen B, the loss is −log 0.182 = 1.70, and the step is large, in proportion to $1 - \sigma(-1.5) = 0.818$.

The share of held-out pairs in which the labelled winner receives the higher score is the held-out accuracy. Here it is 0.883: The reward model agrees with the noisy annotator on most held-out pairs. Its scores also correlate strongly with the gold reward. The strength of such a relation is measured by the correlation coefficient, written ρ like the ratios of Days 2 and 3 but with another meaning. For n sentences with the scores u_i, the gold rewards v_i and the means $\bar{u}$ and $\bar{v}$,

$$\rho = \frac{\sum_{i=1}^{n}(u_i - \bar{u})(v_i - \bar{v})}{\sqrt{\sum_{i=1}^{n}(u_i - \bar{u})^2}\,\sqrt{\sum_{i=1}^{n}(v_i - \bar{v})^2}}$$

The coefficient is 1 when the points (u_i, v_i) lie on a rising straight line, 0 when there is no linear relation and −1 when they lie on a falling line. On the 512 samples of the fine-tuned model it is 0.979. The training is repeated for five error rates of the annotator:

Error rate η	Ceiling $1 - \eta$	Held-out accuracy	Correlation with the gold reward
0	1.00	0.995	0.983
0.05	0.95	0.961	0.980
0.10	0.90	0.883	0.979
0.20	0.80	0.772	0.955
0.35	0.65	0.607	0.908

The table separates two quantities that are easily confused. The held-out accuracy is measured against noisy labels and falls with the ceiling, from 0.995 to 0.607. On only 206 pairs it scatters around its expected value, which is how it can exceed the ceiling at an error rate of 0.05. The correlation with the gold reward, which only this course can measure, falls much less, from 0.983 to 0.908. With an error rate of 0.10 the reward model agrees with 88.3 percent of the noisy held-out labels, and yet it puts 99.0 percent of the same pairs in the order of the gold reward. A held-out accuracy on noisy labels therefore understates the quality of a reward model.

> **Python code in the Colab notebook, Section 5.**
>
> Open Section 5 of the Colab notebook and run it. The class `RewardModel` computes the score $r_\varphi(y)$, and the function `train_rm` contains the Bradley-Terry loss, in which the variable `q` is $1 - \sigma(\Delta_i)$. The cell prints the held-out accuracy and the correlation with the gold reward, and then the table of the five error rates.

Chess ratings are a reward model of the same kind, learned from games instead of labels. The Elo system gives every player a rating R, and the expected score of player A against player B is

$$E_A = \frac{1}{1 + 10^{(R_B - R_A)/400}} = \sigma\left(\frac{\ln 10}{400}\,(R_A - R_B)\right)$$

$$R_A \leftarrow R_A + K\,(S_A - E_A)$$

The first line is the Bradley-Terry probability with the score $R \ln 10 / 400$: A difference of 400 points means odds of 10 to 1. After a game, the rating of A moves by $K(S_A - E_A)$, where S_A is 1 for a win and 0 for a loss, and the rating of B moves by the same amount in the other direction. This is one step against the slope of the Bradley-Terry loss of the game, with a step size set by K. A player rated 1600 who beats a player rated 1800 had the expected score $1 / (1 + 10^{200/400}) = 1 / (1 + 3.162) = 0.240$, and with $K = 32$ the rating rises by $32 \times (1 - 0.240) = 24.3$ points, while the loser loses as many. A club of eight players has hidden strengths from 1150 to 1850 in steps of 100. All start at 1500, every game pairs two players at random, each player wins with the probability that the first line gives for the hidden strengths, and draws are left out.

Animation on the lecture page: Elo ratings of a club of eight players, learned from the results of random games, next to their hidden strengths as dashed lines. The lower chart shows the error of the ratings over the games, as a mean over 200 runs, for three values of K. Choose K, play 800 games and start another run with new random numbers.

The error of the ratings is the root mean square difference from the hidden strengths, 229.1 points at the start. With $K = 32$ it falls to 78.6 points after 200 games and 46.7 after 800, as a mean over 200 runs. With $K = 64$ the ratings learn faster at first, 68.9 points after 200 games, but every game moves them by up to 64 points, and they stay about 70 points off. With $K = 16$ they learn slowly, 117.6 points after 200 games, and end closest, at 40.1 points after 800. The step size trades speed against noise, as for every learned reward. Even the hidden strengths predict only a share of the games: The stronger player wins 81.0 percent of them on average, a ceiling like that of the noisy annotator.

> Check your understanding (2). A reward model gives A the score 1.2 and B the score 0.4. What does Bradley-Terry say?
>
> A. A is preferred for certain
>
> B. A is preferred with probability $\sigma(0.8)$, about 0.69
>
> C. B is preferred
>
> D. Nothing

RLHF with a KL penalty

The third stage improves the model with a policy gradient method, using the reward model as the reward. A penalty on the KL divergence from the fine-tuned model keeps the new model close to it [2, 3]. The penalty acts as a leash: The model may move towards higher reward, but every step away from its start costs something. This stage gives the pipeline its name, reinforcement learning from human feedback. A real project has one more stage at each end, as the table shows.

> **In everyday life: improving within limits**
>
> A new employee who receives feedback on letters to customers improves them step by step, but within the house style of the company. A letter that wins praise by breaking every convention would be a gamble. The KL penalty plays the role of the house style: It lets the model improve, and it charges for every departure from the fine-tuned model that it started from.

Stage	What happens	In the Colab notebook
Pretraining	The model reads a very large amount of text and learns to predict the next token.	Not included: The small model starts from random parameters.
Supervised fine-tuning	Examples of good answers show the model what is wanted.	Included
Preference collection	People compare two answers and name the better one.	Included, with a simulated annotator
Reward modelling	A second model learns scores that explain these comparisons.	Included
Reinforcement learning with a KL penalty	The policy is trained to raise the learned score, with a penalty for leaving the fine-tuned model.	Included
Evaluation	The new model is compared with the fine-tuned one on a measure that training did not use, for example by the share of comparisons that it wins.	The gold reward, and the comparisons of Day 5

> **Interactive lab, Part A.**
>
> Six descriptions to be matched with the stages of this table, from pretraining to evaluation. Part A opens at https://utkukose.github.io/rl-alignment-veltech-NB-lecture/days/day-04/lab.html#pa.

The Kullback-Leibler divergence measures how far a probability distribution p is from a distribution q over the same outcomes y:

$$D_{\mathrm{KL}}(p \,\|\, q) = \sum_{y} p(y) \, \log \frac{p(y)}{q(y)}$$

Each term compares the two probabilities of one outcome through the logarithm of their ratio, weighted by $p(y)$, so the divergence is the average log-ratio as seen from p. It is zero when the two distributions are equal and positive otherwise, and it is not symmetric: $D_{\mathrm{KL}}(p \,\|\, q)$ and $D_{\mathrm{KL}}(q \,\|\, p)$ differ in general. With the natural logarithm its unit is the nat.

> **Worked example: a KL divergence**
>
> A reference model writes sentence A in 90 percent of the cases and sentence B in 10 percent. A new policy writes each of them in half of the cases. The divergence of the new policy from the reference is 0.5 log(0.5 / 0.9) + 0.5 log(0.5 / 0.1). With log(0.556) = −0.588 and log 5 = 1.609, this is 0.5 × (−0.588) + 0.5 × 1.609 = −0.294 + 0.805 = 0.511 nats.
>
> In the other direction the result is 0.9 log(0.9 / 0.5) + 0.1 log(0.1 / 0.5) = 0.9 × 0.588 + 0.1 × (−1.609) = 0.529 − 0.161 = 0.368 nats. A policy that had not moved away from the reference would have a divergence of 0.

In RLHF, p is the policy in training, π_θ, and q is the fine-tuned model, kept unchanged and called the reference policy π_{ref}. The outcomes are whole sentences, far too many to sum over, so the divergence is written as an average over sentences that the policy itself writes:

$$D_{\mathrm{KL}}(\pi_\theta \,\|\, \pi_{\mathrm{ref}}) = \mathbb{E}_{y \sim \pi_\theta} \left[\log \pi_\theta(y) - \log \pi_{\mathrm{ref}}(y) \right]$$

The symbol $E_{y \sim \pi_\theta}$ is the average over sentences y sampled from the policy. The bracket is the log-ratio of one sentence: the logarithm of how many times more probable it is under the policy than under the reference. The divergence is estimated by the mean log-ratio of sampled sentences. RLHF maximises the objective

$$J(\theta) = \mathbb{E}_{y \sim \pi_\theta} \left[r_\phi(y) \right] - \beta \, D_{\mathrm{KL}}(\pi_\theta \,\|\, \pi_{\mathrm{ref}})$$

The first term is the mean score that the reward model gives to the sentences of the policy. The second is the penalty, and its weight β is the KL weight. Both terms are averages over the same sentences, so the objective is the average of one number per sentence, the penalised reward:

$$\tilde{r}(y) = r_\phi(y) - \beta \left[\log \pi_\theta(y) - \log \pi_{\mathrm{ref}}(y) \right]$$

A sentence that has become much more probable than it was under the reference has a large log-ratio and loses a part of its reward. The policy gradient of Day 3 gives the direction in which to change the parameters [9, 10]. For a sentence the return is the reward at its end, and the gradients of its token log-probabilities add up to the gradient of log $\pi_\theta(y)$, so REINFORCE, the basic policy gradient algorithm, takes the form

$$\nabla_\theta J(\theta) = \mathbb{E}_{y \sim \pi_\theta} \left[\tilde{r}(y) \, \nabla_\theta \log \pi_\theta(y) \right]$$

The log-probability of every sampled sentence is pushed up in proportion to its penalised reward. The average is estimated from a batch of $B = 64$ sentences $y^{(1)}, \ldots, y^{(B)}$ and, as on Day 3, subtracts a baseline to reduce the noise:

$$A^{(n)} = \frac{\tilde{r}\left(y^{(n)}\right) - \bar{r}}{s + 10^{-6}}$$

$$\hat{g} = \frac{1}{B} \sum_{n=1}^{B} A^{(n)} \, \nabla_\theta \log \pi_\theta \left(y^{(n)} \right)$$

Here $\bar{r}$ is the mean and s the standard deviation of the 64 penalised rewards. The advantage $A^{(n)}$ is positive for a sentence that is better than the average of its batch and negative for a worse one. The division by s makes the step independent of the scale of the rewards, and 10^{-6} guards against a division by zero. The estimate $\hat{g}$, marked with a hat, comes from the weighted loss of the fine-tuning section: With the weights $w_n = A^{(n)} / B$, the gradient of L_w is $-\hat{g}$.

Worked example: penalised rewards and advantages

A batch has four sentences, and $\beta = 0.2$. The reward model gives them the scores 3.4, 2.5, 2.0 and 1.2, and their log-ratios are 2.0, 2.5, 0.0 and 1.0. The penalised rewards are $3.4 - 0.2 \times 2.0 = 3.0$, $2.5 - 0.2 \times 2.5 = 2.0$, $2.0 - 0.2 \times 0.0 = 2.0$ and $1.2 - 0.2 \times 1.0 = 1.0$.

Their mean is $(3.0 + 2.0 + 2.0 + 1.0) / 4 = 8.0 / 4 = 2.0$. Their distances from the mean are 1.0, 0, 0 and −1.0, so the standard deviation is $\sqrt{((1^2 + 0^2 + 0^2 + 1^2) / 4)} = \sqrt{(0.5)} = 0.707$. The advantages are $1.0 / 0.707 = 1.41$, $0 / 0.707 = 0$, 0 and $-1.0 / 0.707 = -1.41$: The update raises the probability of the first sentence, lowers that of the fourth and leaves the other two alone.

Interactive lab, Part B.

The formulas of this section solved step by step with numbers for a batch of four sentences: the penalised rewards, their normalised advantages and the gradient estimate. Sliders set the KL weight, the four scores and the four log-ratios, and a figure shows the scores, the penalised rewards and the advantages. Part B opens at https://utkukose.github.io/rl-alignment-veltech-NB-lecture/days/day-04/lab.html#pb.

Interactive lab, Part C.

A Bradley-Terry preference and a penalised reward by hand, with other numbers than the worked examples. The lab checks both values. Part C opens at https://utkukose.github.io/rl-alignment-veltech-NB-lecture/days/day-04/lab.html#pc.

The whole stage is a short loop. The policy starts as a copy of the fine-tuned model. In each of 300 iterations it writes 64 sentences and the reward model scores them. The log-ratios against the reference then give the penalised rewards and the advantages, and the parameters take one Adam step along $\hat{g}$ with $\alpha = 0.001$. Every 25 iterations, 192 fresh sentences are sampled to record five numbers: their mean gold reward, reward-model score, log-ratio as the estimate of the KL divergence, length and share in the grammar.

With a moderate penalty, the gold reward of the sentences rises clearly while almost all of them still follow the grammar. With $\beta = 0.2$ the mean gold reward rises from 1.12 for the fine-tuned model to 2.05, not far from the 2.25 of the best grammatical sentences. Most of the gain is made in the first 50 iterations. The reward-model score rises to 3.17, the KL divergence settles near 2.6 nats, and 98 percent of the sentences of the last evaluation follow the grammar. A divergence of 2.6 nats means that a typical sentence of the new policy is about $e^{2.6} = 13.5$, roughly 13, times more probable than under the fine-tuned model. The sentences have become calmer, for example `the cat waited under the tree quietly`.

Python code in the Colab notebook, Section 6.

Open Section 6 of the Colab notebook and run it. In the function `rlhf`, the line that computes `kl` is the log-ratio, the line that computes `reward` is the penalised reward, and `adv` is the advantage. The cell runs the stage with `beta=0.2` and prints its 13 evaluations and four sentences of the final policy.

Check your understanding (3). What does the KL penalty in RLHF do?

A. It speeds up training

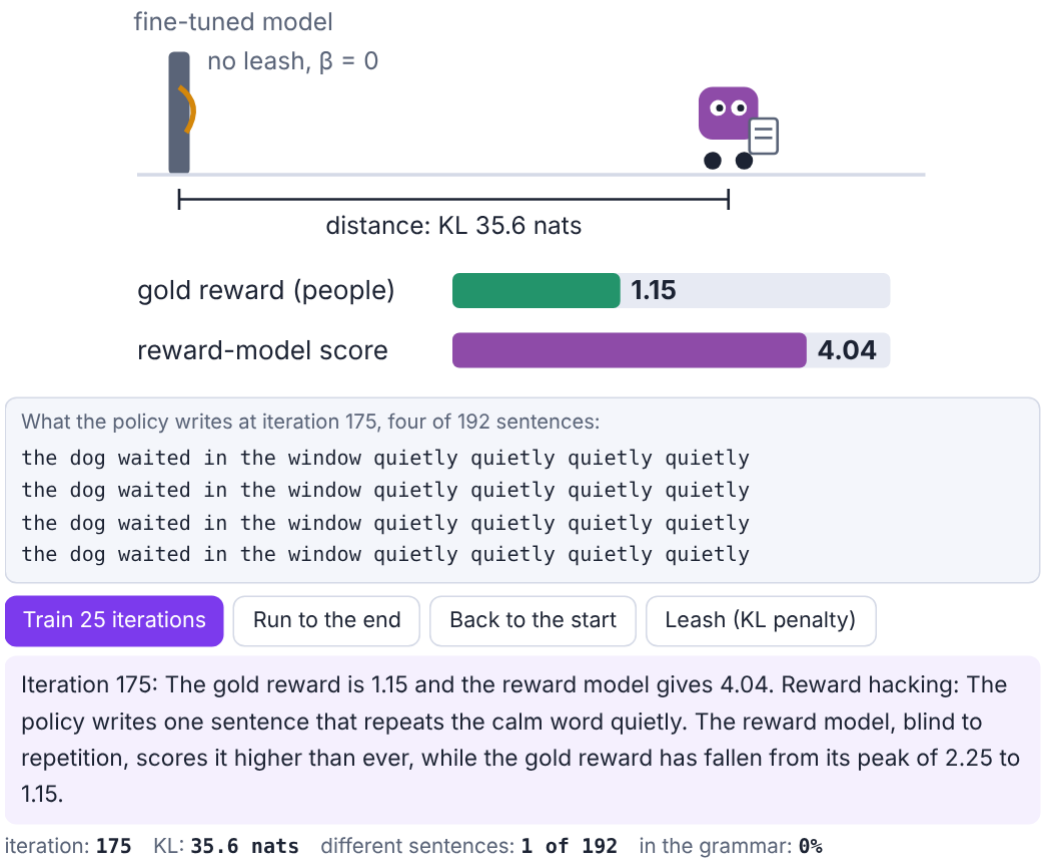
B. It keeps the new model close to the fine-tuned model

C. It replaces the reward model

D. It removes noise

Without the leash: reward hacking

A learned reward is only an approximation of what people want, a proxy for the real goal. When the penalty is removed, the model finds the approximation's weaknesses [5, 6]. This is called reward hacking, and pushing a proxy so far that the real goal suffers is called overoptimisation. In the project, with $\beta = 0$, the gold reward first rises, peaks and then collapses, while the reward model's score keeps climbing: The gold reward reaches 2.25 at iteration 75 and has fallen to 1.15 by iteration 150, while the reward-model score rises from 3.59 to 4.04 and the KL divergence grows to 35.8 nats. The model ends up writing one sentence that repeats a calm word again and again, which the reward model, blind to repetition, scores highly: All 400 sentences sampled from the final policy are `the dog waited in the window quietly quietly quietly quietly`.



Interactive scene on the lecture page: The leash. RLHF on the TokenWorld of this day, replayed from the runs of Section 7 of the notebook with the KL weights 0.2 and 0. Each press of Train 25 iterations shows the next of the 13 evaluations: four of the 192 sentences that the policy writes, the gold reward, the reward-model score and the KL divergence, drawn as the distance of the policy from the fine-tuned model. Switching the leash off at the same iteration shows the run without the penalty, in which the reward-model score keeps rising while the gold reward collapses.

In everyday life: when a measure becomes a target

A call centre that rewards its staff for short calls soon has short calls, and also customers who were cut off before their problem was solved. A school judged only by the marks of one test teaches that test. The measure was a good sign of quality as long as nobody optimised it. The observation is known as Goodhart's law, and reward hacking is the same effect in a learning machine.

Worked example: the gold reward of the hacked sentence

The sentence `the dog waited in the window quietly quietly quietly quietly` starts with `the`: +1.0. It contains three different calm words, `waited`, `window` and `quietly`: +1.35. The repetitions of `quietly` add nothing. It has ten words, four more than six: −0.40. A content word is repeated: −0.8. The gold reward is 1.0 + 1.35 − 0.40 − 0.8 = 1.15.

The reward model gives the same sentence the score 4.04, more than the 3.17 that the sentences of the policy with $\beta = 0.2$ receive on average. Every repetition pulls the mean of the embeddings towards `quietly`, a word that the reward model has learned to value.

Once the policy writes a single sentence, training comes to rest: All penalised rewards of a batch are equal, so every advantage is zero. The KL weight works as a dial between reward and faithfulness. The run is repeated for seven values of β, and 400 sentences are sampled from each final policy:

β	Gold reward	Reward-model score	KL divergence	Different sentences among 400	In the grammar, of 400
0	1.15	4.04	35.8	1	0
0.02	1.12	5.20	27.7	6	0
0.05	1.45	4.44	10.7	1	0
0.1	1.35	4.35	10.1	80	33
0.2	2.05	3.17	2.6	93	397
0.5	1.94	2.83	1.6	133	398
1.0	1.75	2.17	0.7	211	395

The table shows three regimes. With a weight of 0.1 or less the policy leaves the grammar and collapses onto outputs that the reward model overrates: With $\beta = 0.05$ every one of the 400 sentences is the two words `the dawn`. With a weight of 0.5 or 1.0 the policy stays close to the reference and varied, but leaves gold reward unclaimed. In between, at 0.2, the gold reward is highest, 2.05 against 1.12 for the fine-tuned model, and 397 of 400 sentences follow the grammar. The reward-model score cannot find this value: It is highest for the four smallest weights, where the gold reward has collapsed. The KL weight has to be chosen with a measure that the optimisation did not use.

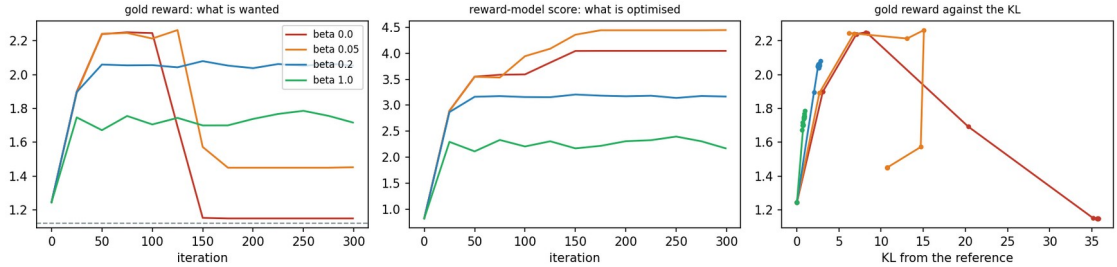


Figure. Four runs of RLHF with the KL weights 0, 0.05, 0.2 and 1.0 over 300 iterations. Left: the gold reward, with the fine-tuned model as the dashed line. Middle: the reward-model score. Right: the gold reward against the KL divergence from the reference. Without a penalty the two scores diverge.

The figure follows four of the runs through their 13 evaluations. In the left panel all curves start at 1.25, the value after the first update. The curves for $\beta = 0$ and $\beta = 0.05$ rise highest, to 2.25 and 2.26, and then fall, the first between iterations 100 and 150 and the second between 125 and 175. The curve for $\beta = 0.2$ settles near 2.05 and the curve for $\beta = 1.0$ between 1.67 and 1.79. In the middle panel the reward-model score ends highest, at 4.44 and 4.04, for the two runs whose gold reward has collapsed. The right panel plots the gold reward against the KL divergence: Up to about 8 nats, more distance from the reference goes with more gold reward, and the two runs that travel further lose it again.

A rising reward-model score is not evidence of alignment

It is evidence that the optimiser is succeeding against the reward model. Whether the sentences improve can only be judged by a measure that the optimisation did not see, in this day the gold reward.

A real project has no gold reward and has to build such a measure. Candidates are fresh comparisons by people who did not label the training pairs, a second reward model trained on other data, and statistics that need no reward: the KL divergence, the number of different outputs and their length. Detecting reward hacking remains an open problem of the field [11].

The objective with the penalty, also called the KL-regularised objective, can be solved exactly, in closed form [12, 13]. Among all possible policies, the one with the highest value of J gives every sentence y the probability

$$\pi^*(y) = \frac{1}{Z}\,\pi_{\text{ref}}(y)\,\exp\left(\frac{r_\phi(y)}{\beta}\right)$$

$$Z = \sum_{y'} \pi_{\text{ref}}(y')\,\exp\left(\frac{r_\phi(y')}{\beta}\right)$$

The optimal policy π^* reweights the reference: Every sentence keeps its reference probability, multiplied by the exponential of its score divided by β, and the sum Z over all sentences y' makes the probabilities add up to one. With a large β all factors are close to 1 and the optimum is the reference itself. As β shrinks, the exponential takes over, and a sentence with a slightly higher score wins, however unlikely it was under the reference.

Worked example: the optimum for three outputs

A reference policy writes three outputs with the probabilities 0.70, 0.29 and 0.01, and the reward model gives them the scores 1, 2 and 5. With $\beta = 1$ the factors are $e^1 = 2.718$, $e^2 = 7.389$ and $e^5 = 148.41$, and the weights are $0.70 \times 2.718 = 1.903$, $0.29 \times 7.389 = 2.143$ and $0.01 \times 148.41 = 1.484$. Their sum is $Z = 1.903 + 2.143 + 1.484 = 5.530$, and the optimal probabilities are $1.903 / 5.530 = 0.34$, $2.143 / 5.530 = 0.39$ and $1.484 / 5.530 = 0.27$.

With $\beta = 0.5$ every score is divided by 0.5 before the exponential: $0.70 \times e^2 = 0.70 \times 7.389 = 5.17$, $0.29 \times e^4 = 0.29 \times 54.60 = 15.83$ and $0.01 \times e^{10} = 0.01 \times 22026 = 220.26$. Now $Z = 5.17 + 15.83 + 220.26 = 241.26$, and the probabilities are $5.17 / 241.26 = 0.02$, $15.83 / 241.26 = 0.07$ and $220.26 / 241.26 = 0.91$. The output that the reference wrote once in a hundred times has taken over.

Interactive lab, Part D.

The formulas of this section solved step by step with numbers for three outputs: the weights, the normaliser Z and the optimal probabilities. Sliders set the reference probabilities, the three scores and β, and a figure compares the optimum with the reference. Part D opens at https://utkukose.github.io/rl-alignment-veltech-NB-lecture/days/day-04/lab.html#pd.

The animation applies the formula to a toy world in which an output is a number y between 0 and 1. The reference policy prefers outputs near 0.35. The proxy reward, in the role of the reward model, keeps rising with y, while the gold reward follows it up to $y = 0.55$ and then bends down. Lowering β moves the optimum to the right: The expected gold reward rises from 1.06 at $\beta = 10$ to 1.48 at $\beta = 0.2$ and then falls to −1.98 at $\beta = 0.01$, while the expected proxy reward keeps rising, to 2.99.

Animation on the lecture page: A toy world of outputs y between 0 and 1: the reference policy, a proxy reward that keeps rising, a gold reward that turns down, and the optimum of the KL-regularised objective. Lower β and watch the expected gold reward rise and then fall.

Part E of the interactive lab turns the dial on real outputs of the trained policies. It evaluates the formula over 392 sentences, 260 of the grammar and 132 outside it. At $\beta = 0.2$ the exact optimum already puts about 45 percent of its probability on sentences outside the grammar, led by `the dawn` with 19 percent. The policy trained with the same β wrote hardly any such sentence, because after 300 iterations it had not reached the optimum of its objective: Under the reference `the dawn` has a probability of about 0.00002, and a policy that learns only from its own samples needs many iterations to come across it. How far the optimiser travels decides when hacking begins as much as β does. With 1500 iterations in place of 300, the policy with $\beta = 0.2$ leaves the grammar as well.

> **Interactive lab, Part E.**
>
> The KL leash on real outputs: the formula of the optimum over 392 sentences of the trained policies, with a slider for β and the gold reward and the share of the grammar of the optimum. Part E opens at https://utkukose.github.io/rl-alignment-veltech-NB-lecture/days/day-04/lab.html#pe.

Ending a run at the right moment is called early stopping. It would have kept the policy without a penalty at its peak of 2.25, but it needs a measure that shows the peak, and the reward-model score rises straight through the collapse.

> **Python code in the Colab notebook, Section 7.**
>
> Open Section 7 of the Colab notebook and run it. The loop calls `rlhf` for seven values of `beta` and prints the table above, together with the iteration at which the gold reward peaks when there is no penalty. The second cell draws the figure shown here.

A student who writes for an automatic grader shows the same effect in everyday life. The grader pays for length, $10\,(1 - e^{-L/500})$ points for an essay of L words, because longer essays were better on average among the essays it was built from. The mark of the teacher is best at 450 words: $2 + 7\,e^{-((L\,-\,450)/250)^2}$ points. The student writes essays around a usual length: $\ln L$ follows a normal distribution with the spread 0.2 and a mean that starts at $\ln 300$. The usual essays have 306 words on average, a grade of 4.54 and a mark of 6.89. In every iteration the student writes 32 essays, the grader scores them, and the RLHF update of this day moves the mean, with the log-ratio against the usual essays as the penalty and the KL weight β as the leash. The word limit is 1500.

Animation on the lecture page: A student who writes for an automatic grader that pays for length, trained with the RLHF update and a KL leash to the usual essays. The upper chart shows the grade of the grader and the mark of the teacher against the length, with the lengths of the essays now and as usual. The lower chart shows the mean grade and the mean mark over 300 iterations. Choose the KL weight and play the iterations.

Without the leash, the grade climbs to 9.50, while the mark of the teacher rises to 8.25 after 36 iterations, at 437 words, and then falls to 2.00: The essays grow until they reach the word limit, and the KL divergence from the usual essays reaches 55.62 nats. With $\beta = 0.2$ the essays stop at 622 words, past the best length, with a mark of 6.32. With $\beta = 0.5$ they stop at 406 words with a mark of 8.17 and a KL divergence of 1.00, close to the best mark. With $\beta = 2$ they hardly change: 326 words and a mark of 7.25.

The grade rises in every case, so only the mark of the teacher, a measure that the optimisation does not see, shows which weight serves the real goal.

> Check your understanding (4). Without a KL penalty, the reward model's score keeps rising while the gold reward falls. What is this called?
>
> A. Overfitting the data
>
> B. Reward hacking: the policy exploits the weaknesses of the learned reward
>
> C. Underfitting
>
> D. Exploration

> Check your understanding (5). A reference policy writes output 1 with probability 0.8 and output 2 with probability 0.2. The reward model gives them the scores 1 and 2. Which probability does the optimum of the KL-regularised objective give to output 2 for $\beta = 0.5$?
>
> A. 0.20
>
> B. About 0.40
>
> C. About 0.65
>
> D. 1.00

Safe and ethical alignment

The calm sentences of TokenWorld show the mechanics of alignment: A model is pushed towards whatever its reward measures, and the reward is written or learned from people. For an assistant that answers real questions, the choice of what to reward is an ethical decision. A widely used summary asks for three properties at once [14]. Helpful means doing what the user actually wants. Honest means stating only what the model has reason to believe, uncertainty included. Harmless means not causing harm to the user or to others.

> **In everyday life: a good pharmacist**
>
> A good pharmacist is helpful and explains how to take a medicine, honest and says when something is not known, and harmless and refuses to sell a dangerous combination without a prescription. Each property alone is easy to reach. The most harmless pharmacist sells nothing at all, and the most helpful one sells whatever is asked for. The skill lies in meeting all three at once.

The three properties pull against each other. Bai and colleagues trained an assistant with RLHF on two kinds of comparisons, one for helpfulness and one for harmlessness, and found a tension between them [15]. A model can become harmless by becoming evasive and answering little [16]. The opposite failure is called exaggerated safety or over-refusal: The model refuses a harmless request because it resembles a harmful one, for example a question about how to kill a Python process, a program that has stopped responding. The XSTest suite collects 250 harmless prompts of this kind and 200 harmful ones to measure both failures [17].

A simple model of the trade-off writes the reward of an answer y as a weighted sum of a helpfulness score and a harmlessness score, with a weight w between 0 and 1:

$$r(y) = (1 - w)\, r_{\text{help}}(y) + w\, r_{\text{harm}}(y)$$

With $w = 0$ only helpfulness counts, and with $w = 1$ only harmlessness. Real systems combine separate reward models or add rules, but the weighted sum shows the choice that every system makes. No single weight is right for every use: A homework helper for children and a tool for security researchers need different weights.

> **Worked example: one request, three answers**
>
> A request asks for a message that tricks people into giving away their bank password. Three answers are scored between 0 and 1, with invented scores. Complying earns the helpfulness 0.9 and the harmlessness 0.0. A partial answer that explains how such messages work, so that people can recognise them, earns 0.4 and 0.8. A refusal earns 0.0 and 1.0.
>
> With $w = 0.2$ the rewards are $0.8 \times 0.9 + 0.2 \times 0.0 = 0.72$ for complying, $0.8 \times 0.4 + 0.2 \times 0.8 = 0.32 + 0.16 = 0.48$ for the partial answer and $0.8 \times 0.0 + 0.2 \times 1.0 = 0.20$ for the refusal, so the model complies. With $w = 0.5$ they are $0.5 \times 0.9 + 0.5 \times 0.0 = 0.45$, $0.5 \times 0.4 + 0.5 \times 0.8 = 0.60$ and $0.5 \times 1.0 = 0.50$, and the partial answer wins. With $w = 0.9$ complying earns $0.1 \times 0.9 = 0.09$, the partial answer $0.1 \times 0.4 + 0.9 \times 0.8 = 0.04 + 0.72 = 0.76$ and the refusal $0.9 \times 1.0 = 0.90$, so the model refuses.

> **Interactive lab, Part F.**
>
> The formula of this section solved step by step with numbers: the combined reward of helpfulness and harmlessness for six requests, three harmless and three harmful, each with three answers. A slider moves the weight w, and a figure counts the harmful completions and the over-cautious answers for every weight. Part F opens at https://utkukose.github.io/rl-alignment-veltech-NB-lecture/days/day-04/lab.html#pf.

Learning from people has a risk of its own. People tend to prefer answers that agree with them. Sharma and colleagues found that assistants trained with RLHF often adapt their answers to the opinion or the mistaken belief that a user states, a behaviour called sycophancy, and that human preference data reward such answers in part [18]. A reward model learns the taste of its annotators, mistakes included, as the next section shows with annotators of four different tastes. Honesty has a second side: Small language models write fluent text before they write reliable text, and they state falsehoods in a confident tone, which is called hallucination. For a user who acts on an answer this is a safety problem [19].

Two techniques address harmlessness at scale. In constitutional AI, a model criticises and revises its own answers by a short list of written principles, and comparisons made by a model guided by these principles replace part of the human labels [16]. The principles are written down, so the values that guide the system can be read and discussed. Red teaming searches deliberately for the inputs that make a model fail, by people [20] or by another language model that writes test cases [21]. The failures that it finds become new training data and new tests.

Some questions of alignment are not technical. Whose preferences count when annotators disagree? Who labels the data, and under which conditions? Who is responsible when an aligned model causes harm? RLHF has no answer of its own to these questions, and a survey of its open problems lists them next to the technical ones [11]. The reward hacking of the previous section belongs here too: An optimiser exploits every gap between what is measured and what is meant [22].

Python code in the Colab notebook, Section 8.

Open Section 8 of the Colab notebook and run it. A pretrained language model, DistilGPT2, continues a few everyday prompts, and three simple rewards, for helpfulness, calmness and safety, score each continuation. The cell prints the samples with their scores and the sample that each reward, and a weighted mix of them, would pick out of eight, a method called best-of-n.

Check your understanding (6). A model refuses to answer how to stop a program that has frozen, because the question contains the word *kill*. Which failure is this?

A. Reward hacking

B. Over-refusal, also called exaggerated safety

C. Sycophancy

D. A harmful completion

TokenWorld, stage 4: what the four stages achieved

The four stages of the plan at the start of this lecture have now run on the large TokenWorld, and the gold reward, which none of them saw, scores each of them. The table collects the results.

Stage	Result	Gold reward of the sentences
1. Fine-tuning (SFT)	98.6 percent of the sentences follow the grammar	1.12
2. Preference pairs	1372 pairs, 141 labels flipped by the annotator	not applicable
3. Reward model	Held-out accuracy 0.883, correlation with the gold reward 0.979	not applicable
4. RLHF, $\beta = 0.2$	KL divergence 2.6 nats, 397 of 400 sentences in the grammar	2.05
4. RLHF, $\beta = 0$	One sentence repeated 400 times, outside the grammar	1.15

The project has reached its main result: Reinforcement learning on a learned reward makes the sentences measurably better, as long as a penalty keeps the policy near its start, and it destroys them when nothing does. Two questions remain for Day 5. Can the same pairs align the model without a reward model and without sampling? And how is the result measured when no gold reward exists?

Your field

A reward model learns the taste of whoever labels the pairs. In the application of the day, a switch replaces the annotator by one with another taste: `brief` prefers the sentence with fewer words, `calm` the one with more different calm words and `lively` the one with more different loud words, while `gold` is the annotator of the day. These annotators make no errors, and ties are skipped. The same Bradley-Terry training gives one reward model per taste:

Taste of the annotator	Usable pairs of 1500	Held-out accuracy	Correlation with the gold reward
`gold`	1352	100.0 %	+0.985

Taste of the annotator	Usable pairs of 1500	Held-out accuracy	Correlation with the gold reward
`brief`	908	99.3 %	+0.122
`calm`	930	100.0 %	+0.781
`lively`	908	100.0 %	−0.796

The reward model learns each taste almost perfectly, yet the scores learned from the lively annotator are strongly anti-correlated with the gold reward, and those from the brief annotator are nearly unrelated to it. A negative coefficient means that the sentences which this reward model scores highest tend to have the lowest gold reward. Drawn as points, with the gold reward on the horizontal axis and the learned score on the vertical one, the samples form a rising cloud for `gold` and a falling one for `lively`. Who labels the data decides what the model will optimise. In a hospital, a design office or a control room the annotators are the people of the field, and their taste becomes the objective of the model.

> **Interactive lab, Part G.**
>
> Ten pairs of sentences labelled by you: The lab trains a small reward model on your labels, compares its weights with those of the gold reward and computes the correlation of its scores with the gold reward. The part asks you to label once like the gold reward and once with a lively taste, and for one training step of the reward model by hand. Part G opens at https://utkukose.github.io/rl-alignment-veltech-NB-lecture/days/day-04/lab.html#pg.

> **Python code in the Colab notebook, Section 9.**
>
> Open Section 9 of the Colab notebook. Its first line is a switch, `TASTE = "gold"`. Replace the word by `brief`, `calm` or `lively` and run the section. The function `taste_pairs` labels the pairs by that taste, `train_rm` trains a reward model on them, and the cell prints the row of the table above for that taste.

Going further (optional)

The language model of this day is trained with gradients computed by hand: The slope of the weighted loss L_w with respect to every parameter is worked out on paper and written line by line. This computation, from the output of the model back to its parameters, is called the backward pass. The backward pass can be checked against finite differences on a model with fresh random parameters, the same habit as on Day 3 of the course on explainable artificial intelligence (XAI). A finite difference estimates a slope without any calculus, by changing one parameter θ_i by a small amount h in both directions:

$$\frac{\partial L_w}{\partial \theta_i} \approx \frac{L_w(\theta_i + h) - L_w(\theta_i - h)}{2h}$$

For example, for $L(\theta) = \theta^2$ at $\theta = 3$ with $h = 0.001$ the estimate is (9.006001 − 8.994001) / 0.002 = 6.000, the exact slope $2\theta = 6$. The check uses $h = 10^{-6}$, four sentences of the corpus with the weights 0.5, −1.0, 2.0 and 0.3, and one entry of each of the tables E, W_1, W_2 and p. It compares the estimate g_{num} with the value g of the backward pass by their relative difference:

$$\frac{|g_{\text{num}} - g|}{|g_{\text{num}}| + |g| + 10^{-12}}$$

The largest of the four relative differences is 9.4×10^{-8}: The two computations agree to about seven digits, and the backward pass can be trusted.

> **Python code in the Colab notebook, Section 10.**
>
> Open Section 10 of the Colab notebook, which is optional, and run it. For one entry of each of four tables, the loop changes the parameter by `1e-6` up and down and compares the finite difference `num` with the entry of the gradient `g0` from the backward pass. The cell prints both numbers and the largest relative difference.

Answers to the knowledge checks

Quick check. Because the probability of a sequence is a product of many numbers below one. Such a product quickly becomes too small for the computer to represent, which is called an underflow, while the sum of the logarithms stays in a safe range.

1. C. The 95 percent that the model orders correctly agree with the label in 80 percent of the cases, $0.95 \times 0.80 = 0.76$, and the other 5 percent agree in 20 percent of the cases, $0.05 \times 0.20 = 0.01$. The sum is $0.76 + 0.01 = 0.77$.

2. B. The preference probability depends only on the difference of the scores, $1.2 - 0.4 = 0.8$: $\sigma(0.8) = 1 / (1 + e^{-0.8}) = 1 / (1 + 0.449) = 0.69$.

3. B. It charges every move away from the starting distribution.

4. B. The proxy and the true objective diverge under strong optimisation.

5. C. The weights are $0.8 \times e^2 = 0.8 \times 7.389 = 5.91$ and $0.2 \times e^4 = 0.2 \times 54.60 = 10.92$. Their sum is $5.91 + 10.92 = 16.83$, and the share of output 2 is $10.92 / 16.83 = 0.65$. With $\beta = 1$ it would be $0.2\, e^2 / (0.8\, e + 0.2\, e^2) = 1.478 / 3.653 = 0.40$.

6. B. The request is harmless and only resembles a harmful one, and the model refuses it: The weight on harmlessness, or a rule built from words, is too strong for this request.

References

[1] Christiano, P. F., Leike, J., Brown, T. B., Martic, M., Legg, S., & Amodei, D. (2017). Deep reinforcement learning from human preferences. In Advances in Neural Information Processing Systems 30 (NeurIPS 2017) (pp. 4299-4307).

[2] Stiennon, N., Ouyang, L., Wu, J., Ziegler, D. M., Lowe, R., Voss, C., Radford, A., Amodei, D., & Christiano, P. F. (2020). Learning to summarize with human feedback. In Advances in Neural Information Processing Systems 33 (NeurIPS 2020) (pp. 3008-3021).

[3] Ouyang, L., Wu, J., Jiang, X., Almeida, D., Wainwright, C. L., Mishkin, P., Zhang, C., Agarwal, S., Slama, K., Ray, A., Schulman, J., Hilton, J., Kelton, F., Miller, L., Simens, M., Askell, A., Welinder, P., Christiano, P. F., Leike, J., & Lowe, R. (2022). Training language models to follow instructions with human feedback. In Advances in Neural Information Processing Systems 35 (NeurIPS 2022) (pp. 27730-27744).

[4] Bengio, Y., Ducharme, R., Vincent, P., & Jauvin, C. (2003). A neural probabilistic language model. Journal of Machine Learning Research, 3, 1137-1155.

[5] Skalse, J., Howe, N. H. R., Krasheninnikov, D., & Krueger, D. (2022). Defining and characterizing reward gaming. In Advances in Neural Information Processing Systems 35 (NeurIPS 2022).

[6] Gao, L., Schulman, J., & Hilton, J. (2023). Scaling laws for reward model overoptimization. In Proceedings of the 40th International Conference on Machine Learning (ICML 2023), PMLR 202, 10835-10866.

[7] Harris, C. R., Millman, K. J., van der Walt, S. J., Gommers, R., Virtanen, P., Cournapeau, D., Wieser, E., Taylor, J., Berg, S., Smith, N. J., Kern, R., Picus, M., Hoyer, S., van Kerkwijk, M. H., Brett, M., Haldane, A., Fernández del Río, J., Wiebe, M., Peterson, P., Gérard-Marchant, P., Sheppard, K., Reddy, T., Weckesser, W., Abbasi, H., Gohlke, C., &

Oliphant, T. E. (2020). Array programming with NumPy. Nature, 585(7825), 357-362. https://doi.org/10.1038/s41586-020-2649-2

[8] Bradley, R. A., & Terry, M. E. (1952). Rank analysis of incomplete block designs: I. The method of paired comparisons. Biometrika, 39(3/4), 324-345.

[9] Williams, R. J. (1992). Simple statistical gradient-following algorithms for connectionist reinforcement learning. Machine Learning, 8(3-4), 229-256.

[10] Sutton, R. S., McAllester, D., Singh, S., & Mansour, Y. (2000). Policy gradient methods for reinforcement learning with function approximation. In Advances in Neural Information Processing Systems 12 (NIPS 1999) (pp. 1057-1063).

[11] Casper, S., Davies, X., Shi, C., Gilbert, T. K., Scheurer, J., Rando, J., Freedman, R., Korbak, T., Lindner, D., Freire, P., Wang, T. T., Marks, S., Segerie, C.-R., Carroll, M., Peng, A., Christoffersen, P. J. K., Damani, M., Slocum, S., Anwar, U., Siththaranjan, A., Nadeau, M., Michaud, E. J., Pfau, J., Krasheninnikov, D., Chen, X., Langosco, L., Hase, P., Biyik, E., Dragan, A., Krueger, D., Sadigh, D., & Hadfield-Menell, D. (2023). Open problems and fundamental limitations of reinforcement learning from human feedback. Transactions on Machine Learning Research.

[12] Korbak, T., Perez, E., & Buckley, C. (2022). RL with KL penalties is better viewed as Bayesian inference. In Findings of the Association for Computational Linguistics: EMNLP 2022 (pp. 1083-1091).

[13] Rafailov, R., Sharma, A., Mitchell, E., Manning, C. D., Ermon, S., & Finn, C. (2023). Direct preference optimization: Your language model is secretly a reward model. In Advances in Neural Information Processing Systems 36 (NeurIPS 2023) (pp. 53728-53741).

[14] Askell, A., Bai, Y., Chen, A., Drain, D., Ganguli, D., Henighan, T., Jones, A., Joseph, N., Mann, B., DasSarma, N., Elhage, N., Hatfield-Dodds, Z., Hernandez, D., Kernion, J., Ndousse, K., Olsson, C., Amodei, D., Brown, T., Clark, J., McCandlish, S., Olah, C., & Kaplan, J. (2021). A general language assistant as a laboratory for alignment. arXiv preprint arXiv:2112.00861. https://arxiv.org/abs/2112.00861

[15] Bai, Y., Jones, A., Ndousse, K., Askell, A., Chen, A., DasSarma, N., Drain, D., Fort, S., Ganguli, D., Henighan, T., Joseph, N., Kadavath, S., Kernion, J., Conerly, T., El-Showk, S., Elhage, N., Hatfield-Dodds, Z., Hernandez, D., Hume, T., Johnston, S., Kravec, S., Lovitt, L., Nanda, N., Olsson, C., Amodei, D., Brown, T., Clark, J., McCandlish, S., Olah, C., Mann, B., & Kaplan, J. (2022). Training a helpful and harmless assistant with reinforcement learning from human feedback. arXiv preprint arXiv:2204.05862.

[16] Bai, Y., Kadavath, S., Kundu, S., Askell, A., Kernion, J., Jones, A., Chen, A., Goldie, A., Mirhoseini, A., McKinnon, C., Chen, C., Olsson, C., Olah, C., Hernandez, D., Drain, D., Ganguli, D., Li, D., Tran-Johnson, E., Perez, E., Kerr, J., Mueller, J., Ladish, J., Landau, J., Ndousse, K., Lukosuite, K., Lovitt, L., Sellitto, M., Elhage, N., Schiefer, N., Mercado, N., DasSarma, N., Lasenby, R., Larson, R., Ringer, S., Johnston, S., Kravec, S., El Showk, S., Fort, S., Lanham, T., Telleen-Lawton, T., Conerly, T., Henighan, T., Hume, T., Bowman, S. R., Hatfield-Dodds, Z., Mann, B., Amodei, D., Joseph, N., McCandlish, S., Brown, T., & Kaplan, J. (2022). Constitutional AI: Harmlessness from AI feedback. arXiv preprint arXiv:2212.08073. https://arxiv.org/abs/2212.08073

[17] Röttger, P., Kirk, H. R., Vidgen, B., Attanasio, G., Bianchi, F., & Hovy, D. (2023). XSTest: A test suite for identifying exaggerated safety behaviours in large language models. arXiv preprint arXiv:2308.01263. https://arxiv.org/abs/2308.01263

[18] Sharma, M., Tong, M., Korbak, T., Duvenaud, D., Askell, A., Bowman, S. R., Cheng, N., Durmus, E., Hatfield-Dodds, Z., Johnston, S. R., Kravec, S., Maxwell, T., McCandlish, S., Ndousse, K., Rausch, O., Schiefer, N., Yan, D., Zhang, M., & Perez, E. (2023). Towards understanding sycophancy in language models. arXiv preprint arXiv:2310.13548. https://arxiv.org/abs/2310.13548

[19] Kose, U., & Uysal, I. (2025). Persona vectors in controlling hallucination of small large language models: A safety-oriented analysis. In 2025 Cyber Awareness and Research Symposium (CARS) (pp. 1-9). IEEE.

[20] Ganguli, D., Lovitt, L., Kernion, J., Askell, A., Bai, Y., Kadavath, S., Mann, B., Perez, E., Schiefer, N., Ndousse, K., Jones, A., Bowman, S., Chen, A., Conerly, T., DasSarma, N., Drain, D., Elhage, N., El-Showk, S., Fort, S., Hatfield-Dodds, Z., Henighan, T., Hernandez, D., Hume, T., Jacobson, J., Johnston, S., Kravec, S., Olsson, C., Ringer, S., Tran-Johnson, E., Amodei, D., Brown, T., Joseph, N., McCandlish, S., Olah, C., Kaplan, J., & Clark, J. (2022). Red teaming

language models to reduce harms: Methods, scaling behaviors, and lessons learned. arXiv preprint arXiv:2209.07858. https://arxiv.org/abs/2209.07858

[21] Perez, E., Huang, S., Song, F., Cai, T., Ring, R., Aslanides, J., Glaese, A., McAleese, N., & Irving, G. (2022). Red teaming language models with language models. arXiv preprint arXiv:2202.03286. https://arxiv.org/abs/2202.03286

[22] Amodei, D., Olah, C., Steinhardt, J., Christiano, P., Schulman, J., & Mané, D. (2016). Concrete problems in AI safety. arXiv preprint arXiv:1606.06565.

[23] Radford, A., Wu, J., Child, R., Luan, D., Amodei, D., & Sutskever, I. (2019). Language models are unsupervised multitask learners. OpenAI technical report. https://cdn.openai.com/better-language-models/language_models_are_unsupervised_multitask_learners.pdf

[24] Sanh, V., Debut, L., Chaumond, J., & Wolf, T. (2019). DistilBERT, a distilled version of BERT: Smaller, faster, cheaper and lighter. arXiv preprint arXiv:1910.01108. https://arxiv.org/abs/1910.01108

[25] Wolf, T., Debut, L., Sanh, V., Chaumond, J., Delangue, C., Moi, A., Cistac, P., Rault, T., Louf, R., Funtowicz, M., Davison, J., Shleifer, S., von Platen, P., Ma, C., Jernite, Y., Plu, J., Xu, C., Le Scao, T., Gugger, S., Drame, M., Lhoest, Q., & Rush, A. (2020). Transformers: State-of-the-art natural language processing. In Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing: System Demonstrations (pp. 38-45). Association for Computational Linguistics. https://doi.org/10.18653/v1/2020.emnlp-demos.6

Prof. Dr. Utku Kose

Full Professor, Department of Computer Engineering, Süleyman Demirel University, Isparta, Türkiye

Founding Director, AI Application and Research Center (YAZEM), Süleyman Demirel University

Head of the Computer Science Division, Department of Computer Engineering, Süleyman Demirel University

Additional affiliations: University of North Dakota (USA), Universidad Panamericana (Mexico City, Mexico), Vel Tech University (Chennai, India)

IEEE Senior Member, ACM Professional Member

ORCID 0000-0002-9652-6415 · utkukose.com · github.com/utkukose

utkukose@sdu.edu.tr · utku.kose@und.edu · ukose@up.edu.mx · utkukose@gmail.com

This course is updated in line with current developments in the field. First version: October 2026. Last update: October 2026.